

A PoC on semantic linting: go or no-go?

WHY, IN THREE LINES

1	The problem	Diagrams render, but not all make sense; every reader downstream, human or AI, fills the gaps.
2	The contribution	A semantic linter finds model smells, gives each diagram a maturity level and gates the hand-off, in the editor and in CI.
3	Measured	AI code from Level 1 sequence diagrams fails ~1 in 3 intended behaviours; from Level 2 up, ~1 in 10.

YOUR TEAM

Team / squad	...
Team contact	<i>name, role</i>
Raised by	<i>round-table attendee</i>
Date	...

Your answer

☐ Go

☐ No-go

☐ Not yet

What would change our mind?

...

IF GO, WE AGREE ON

Scope	Success criteria	Owner & timebox	Next check-in
<i>which team, which diagrams</i>	<i>what must the PoC show?</i>	<i>who leads, how many weeks</i>	<i>when you look at the results</i>

Get started in ten minutes

SIX COMMANDS

```
$ pip install pumllint           # or pipx / uv tool install
$ pumllint diagrams/             # findings, with severity
$ pumllint score diagrams/       # maturity level per diagram
$ pumllint score diagrams/ -f html -o report.html
$ pumllint --list-rules          # everything it checks
$ pumllint lsp                   # findings in your editor
```

WHAT YOU GET BACK

Findings

rule, severity and line, e.g. ACT002 • major

A maturity level

per diagram and per set, with what it takes to reach the next one

An HTML report

one self-contained page, to share or attach to a review

Exit codes for CI

0 clean • 1 findings at or above --fail-on (default major) • 2 usage error

Python 3.11 or later • no runtime dependencies (PyYAML only for a YAML config). Read more in the repository: [README](#) • [docs/findings-and-scores.md](#) • [docs/rule-map.md](#) • [docs/business-processes.md](#)

Fit it to your team

Configure

pumllint.yaml (or .toml / .json) is picked up automatically:

```
rules:
  unnamed-diagram: false
  participant-naming:
    severity: major
  requirement-link:
    pattern: "REQ-\d+"
```

One exception in one diagram: ' pumllint: disable=SEQ006

Automate

GitHub Actions, pinned to a release:

```
- uses: fdurieux/pumllint@v0.33.0
  with:
    command: score
    paths: docs/diagrams
    baseline: maturity.json
    min-level: "2"
```

Or per commit: the pre-commit hooks pumllint and pumllint-score.

Connect

SonarQube

-f sonar writes SonarQube's import format: findings on the same dashboard and in pull requests.

ARIS

the separate aris2pumll tool converts processes to PlantUML; they pass the same gate.

Editors

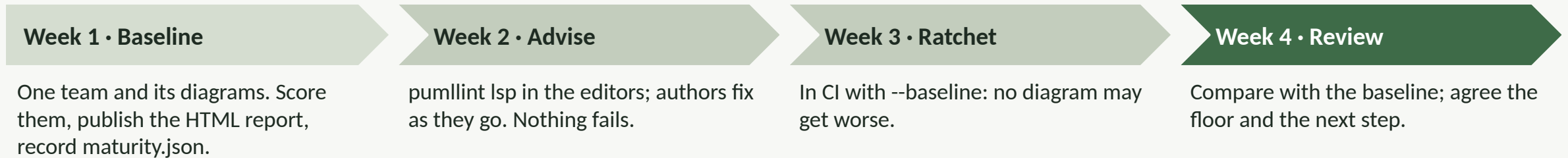
pumllint lsp in any editor that supports LSP; no editor-specific plugins.

Step 1 • ADVISE report only, no gate

Step 2 • RATCHET --baseline maturity.json

Step 3 • FLOOR --min-level 2

A four-week PoC, and what to measure



What to measure

- Findings per dimension, week 1 against week 4
- Sequence and state diagrams at Level 4; processes without a major finding
- Time to fix a finding, as the authors report it
- Noise: findings the team disagrees with, and the config changes they lead to
- Feedback from authors and reviewers
- Rework found downstream: the expected benefit, measured here for the first time

Success criteria

Agreed on the closing slide: what must the PoC show? Write them here before week 1, so week 4 compares against them.

A proposal to adapt: the length, the team and the measures are the PoC owner's call.